

Ejecución de modelos de Machine Learning en sistemas con recursos limitados

Técnicas de Optimización



UNIVERSIDAD
DE
CÓRDOBA

Table of Contents

Abstracto.....	4
Panorama actual.....	4
Diferencias entre TPUs y NPUs.....	5
Desarrollo.....	6
Montaje.....	6
Software.....	7
Sistema complementario.....	8
Especificaciones técnicas de la Coral Edge TPU.....	9
Detección de objetos.....	12
Primera toma de contacto.....	12
Ejecución en CPU.....	14
Ejecución en TPU.....	15
Segmentación de objetos.....	17
Primeros pasos.....	17
Ejecución en CPU.....	18
Benchmarking.....	21
Pipelining.....	23
Entrenamiento de modelo de detección.....	24
Introducción.....	24
Etiquetación de imágenes.....	24
Entrenamiento del modelo.....	28
Verificación del modelo.....	30
Recompilar modelo.....	31
Ejecución de este modelo personalizado.....	32
En CPU.....	32
En TPU.....	33
Entrenamiento de un modelo de segmentación.....	35
Etiquetación de imágenes.....	35
Construyendo dataset en formato compatible.....	37
Entrenamiento del modelo.....	38
Conversión de checkpoint a protobuf.....	39
Conversión de protobuf a TensorFlow Lite.....	40
Problemas encontrados.....	40
Ejecución de modelos en NPU Hailo 8.....	42
Dataflow Compiler.....	42
Evaluación de hardware Hailo.....	43
Conclusión.....	46
Aplicaciones.....	47
Posibles mejoras.....	47
Conclusión.....	48
Glosario.....	49
Bibliografía.....	50

Abstracto

El siguiente trabajo trata de investigar la capacidad que tienen los sistemas empotrados actuales a la hora de realizar tareas que involucran ejecutar tareas de *machine learning* o tareas relacionadas con la inteligencia artificial, obtener una idea base de como se podría mejorar, e investigar opciones para poder mejorarla.

El objetivo de esta investigación es buscar nuevas formas de poder ejecutar este tipo de cargas de tal forma que no tenga un gran impacto en el diseño de estos sistemas, ni tenga un gran impacto en el consumo electrico de estos sistemas, ya que muchos de estos sistemas han de ejecutarse en lugares en los que puede que no tengan acceso a una conexión estable a Internet, o incluso una fuente de electricidad estable o lo suficientemente potente como para acomodar un sistema diseñado para aceptar estas cargas, sin mencionar el costo de esta solución.

Además, en este trabajo se va a explorar el entrenamiento de un modelo de *computer vision* o visión artificial y su eventual exportación a un sistema empotrado.

Panorama actual

Hoy en día son muy comunmente conocidas las GPU o Graphic Processing Unit, que están diseñadas para ejecutar cargas paralelas, gracias a sus miles de nucleos que se pueden activar todos de manera simultánea.

También se puede destacar el *cloud computing*, que es una forma de descargar las tareas más dificiles de procesar por un sistema en un servidor ubicado en otro lugar, pero esto involucra una conexión estable a Internet, en lugares donde a veces no se puede conseguir una conexión lo suficientemente rápida para enviar datos de manera constante.

La última opción, que es menos conocida, son las aceleradoras. Una aceleradora es un dispositivo externo, que dependiendo del modelo y del tipo pueden hacer uso de más o menos energía. Estas opciones, conocidas como TPUs o NPUs, son dispositivos altamente integrados, que a veces pueden encontrarse dentro de una CPU estándar o una GPU moderna, son buenas alternativas debido a las prestaciones que estos presentan para poder ejecutar modelos relacionados con el *machine learning* o *artificial intelligence*.

En este trabajo vamos a hablar de las TPUs. Las TPUs, o *tensor processing unit*. Estos son dispositivos diseñados específicamente para acelerar cargas relacionadas con algoritmos de machine learning.

Esto lo hacen posible gracias a su capacidad de poder realizar operaciones matriciales con una gran velocidad, por medio de dividir estas matrices en vectores, y también por su hardware especializado para realizar este tipo de cargas. Esto hace que se le puedan considerar como procesadores vectoriales, y se caracterizan por su alto ancho de banda y su capacidad de realizar operaciones a alta velocidad de manera paralela.

Existen dos tipos de TPUs, unas son de alta potencia y se pueden encontrar en lugares como centro de datos, y ultimamente incluidas dentro de aglunas GPUs modernas, lo cual están diseñadas para ejecutar algoritmos de mayor exigencia.

El segundo tipo es menos conocido, y son *Edge TPUs*. Estas están diseñadas para trabajar en el borde, *in the edge*, es decir, tienen menos prestaciones que una TPU diseñada para *cloud computing*, o una incluida dentro de una GPU.

Aun así, su ventaja de poder realizar operaciones vectoriales sigue siendo superior que una CPU de un dispositivo que puede encontrarse en sistemas como la *Raspberry Pi*, que no están diseñadas para ejecutar algoritmos de alta potencia y tienen un bajo consumo energético, lo cual las hace idoneas para el trabajo de campo.

Diferencias entre TPUs y NPUs

No existen muchas diferencias en como se comportan un sistema comparado con el otro. En el ámbito común, ambos son *ASICs* o *application specific integrated circuit*, lo cual es un dispositivo diseñado con una tarea en específico. En ambos sistemas, esta tarea es la multiplicación de matrices.

A pesar de ambos compartir la misma funcionalidad, las TPUs son más específicas ya que fueron creadas por Google, para trabajar con su marco de trabajo, que es TensorFlow, mientras que las NPUs se caracterizan por ser un término general, ya que cada una tendrá un entorno de trabajo distinto. Más adelante se podrá ver como es la estructura de las TPUs y también podremos observar un ejemplo de una NPU de la marca Hailo, que ejecuta su propio entorno HailoRT.

Cabe también destacar que ambos se venden bajo la misma funcionalidad, realizar tareas específicas en ambos ámbitos, pero debido a su reciente aparición en el mercado, aún no se ha estandarizado el uso de ambos chips, algunas marcas son más difíciles de programar, no tienen un *SDK* disponible con acceso al público general y pueden ser de mucha dificultad de programar. Actualmente, la API más accesible es la de Google con TensorFlow y TensorFlow Lite en el caso de las TPU, mientras que para acceder a la especificación completa .

Hoy en día se pueden encontrar NPUs integradas dentro los chips más modernos de Apple o como los Windows + Copilot PCs, que hacen uso de las NPUs para poder identificar patrones en el flujo de trabajo de un usuario, y habilitan el procesamiento local de esa información, sin tener una conexión a internet activa permanentemente, e incluso aumentar la privacidad, ya que no se tiene que enviar esa información a servidores de terceros.

Desarrollo

Para la realización de este trabajo, se va a hacer uso de una *Raspberry Pi 3B*, con 1 GB de memoria RAM, 64 GB de almacenamiento en formato microSD, 4 núcleos de CPU *ARM Cortex A-53* y un chip Wi-Fi básico, y se va a ejecutar desde una fuente de alimentación limitada, para poder lograr el mismo efecto que sufriría en una situación no ideal.

Además, como acelerador, vamos a contar con la *Coral TPU*, que es una de las anteriormente mencionadas *Edge TPUs*. Esta ha sido desarrollada por Google y se puede conseguir por 64 euros en el formato USB, pero se puede encontrar más barata en formato M.2 y Mini PCIe.

Finalmente, necesitaremos una cámara, la cual nos permitirá recibir datos en directo que puedan ser procesados por ambos, la CPU dentro de la *Raspberry Pi* y la *Coral Edge TPU*. Para ello, he decidido ir por una *Raspebrry Pi Camera*, ya que se integran muy bien con la *Raspberry Pi* y el sistema operativo *Raspbian*, y tiene librerías que permite interactuar con esta de manera fácil por medio de lenguajes de programación como Python o C++.

Montaje

El montaje consiste en la *Raspberry Pi* con la cámara sujeta encima de ella, lo cual facilita el posicionamiento de la misma para poder apuntarla a distintos objetos, ya que la cámara es muy frágil sin protección y es muy difícil de colocar sin que se mueva.

Al mismo tiempo, como fuente de corriente estoy usando una fuente de ordenador para hacer pruebas, ya que durante la ejecución de los modelos por medio de CPU la máquina se queda corta de energía muy rápidamente.

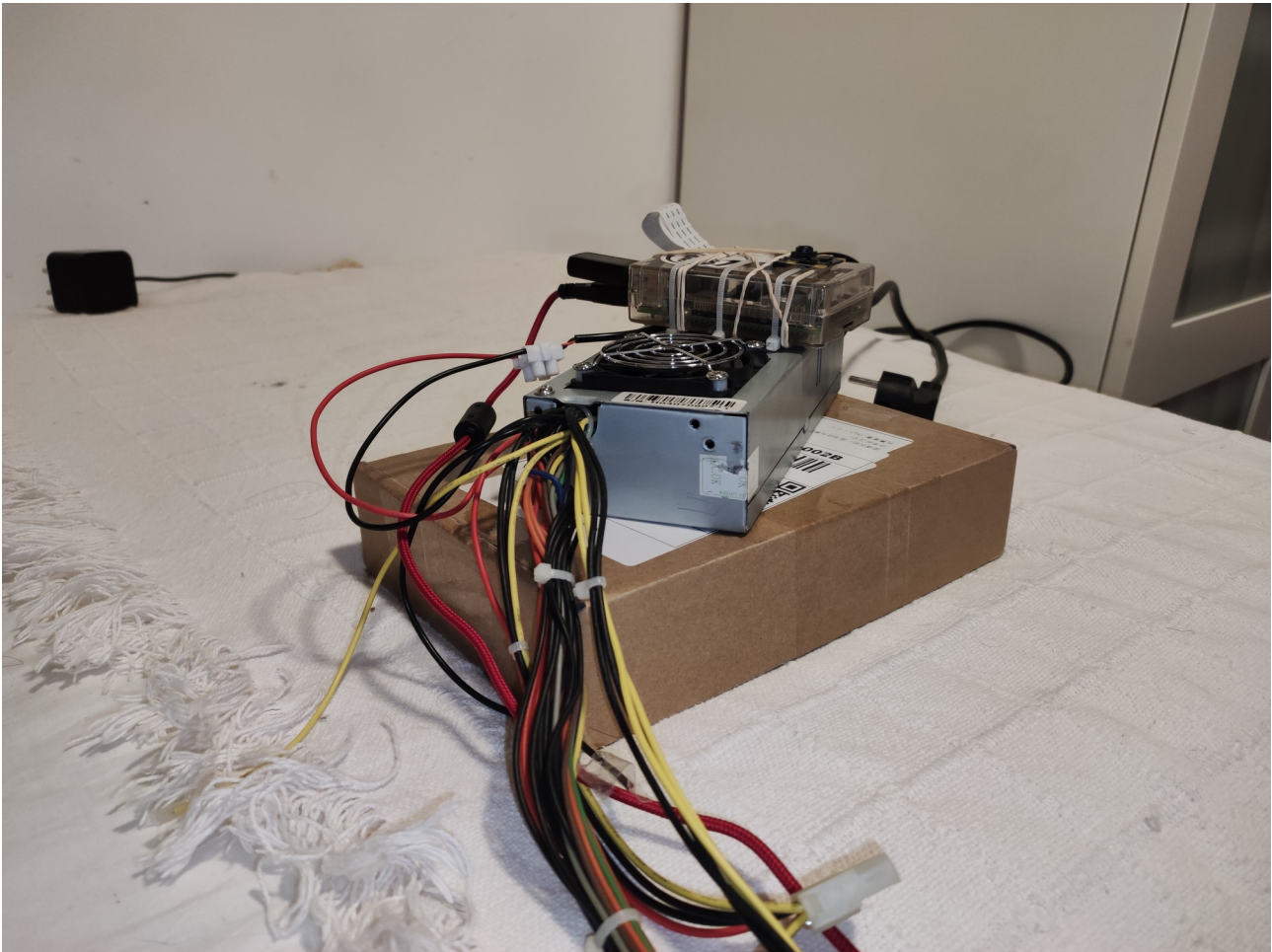


Figure 1: Montaje del sistema completo

El montaje es un poco rudimentario, pero cumple su cometido.

Software

Para el *software* voy a hacer uso de *Raspberry Pi OS* de 64 bit, el sistema es la versión completa con el entorno de escritorio, el cual nos va a poder permitir ver los modelos por pantalla, ya que trae un entorno de escritorio, lo cual facilita la configuración del sistema.

También vamos a hacer mucho uso de *Python 3.11*, que es la versión que trae instalado por defecto el sistema operativo, y de *Anaconda*, que es un gestor de entornos virtuales de *Python*, y se va a utilizar en un sistema auxiliar para administrar los distintos entornos de desarrollo.

En el dispositivo que va a ejecutar los modelos, se va a hacer uso de los *virtual environment* (venv) de *Python* debido a problemas con *Anaconda*, ya que no tiene acceso a ciertas librerías.

Sistema complementario

Durante este trabajo se va a tener que hacer uso de un sistema secundario, el cual va a ser un ordenador sobremesa con las siguientes especificaciones:

- Intel Core i5 9400F
- NVidia GeForce GTX 1660 super
- 16 GB de memoria RAM
- Disco de estado sólido de 512 GB
- Sistema operativo Debian 12 GNU/Linux (bookworm)

El proposito de este sistema es ejecutar tareas de entrenamiento y cuantización, ya que estas tareas, al igual que la compilación, se hacen una sola vez para generar los archivos usados por la TPU, al igual que entrenar y exportar los distintos modelos que se van a usar.

Estas tareas tienen mucha demanda de la GPU del sistema y en algunos casos necesitan una alta cantidad de memoria RAM.

Especificaciones técnicas de la Coral Edge TPU

Hoy en día, Google no ha publicado muchos detalles sobre la Coral Edge TPU, y se desconoce muchos de sus datos, pero se sabe que son una versión reducida que las TPUs que se utilizan en entornos Cloud de Google.

Tanto las Edge TPUs como las TPUs utilizadas en entornos de Cloud de Google comparten muchas características, a diferencia de que las Edge TPUs son ASICs con menor potencia, lo cual los permite ser aplicados para tareas en sistemas empujados.

Es importante empezar definiendo como funciona una red neuronal. Una red neuronal consiste en una serie de neuronas, conectadas una detrás de otra, en las que el resultado de una neurona se conecta a la siguiente neurona, y así sucesivamente. Una neurona contiene los siguientes componentes.

1. Señales de entrada: estas son las entradas que recibe la neurona.
2. Sinapsis: son los pesos de la neurona. Estos pesos modifican la señal de entrada de una forma u otra, pudiendo atenuar la entrada o incrementando su potencia.
3. Acumulador, en la cual se suman los pesos de todas las neuronas, además se puede incorporar un pequeño bias, que podrá modificar este resultado de la suma.
4. Función de activación, la cual se activará si la señal cumulativa que recibe llega a ser mayor que algún threshold establecido.

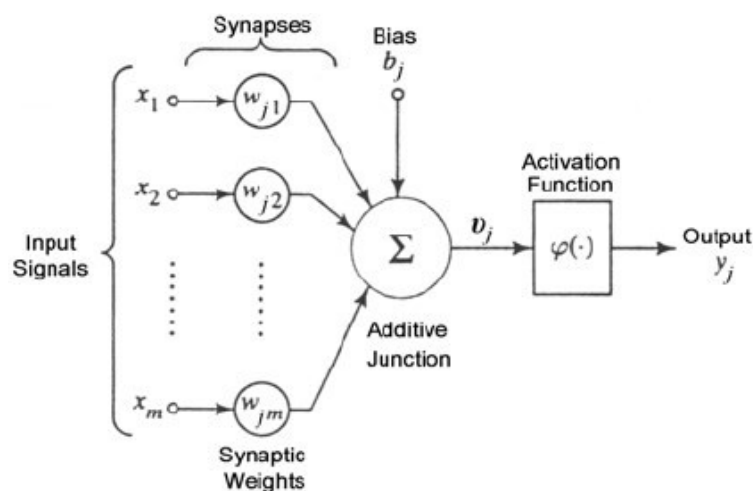


Figure 2: Composición de una neurona

Hoy en día, gracias a los ordenadores que existen y su capacidad de ejecutar código de manera paralela puede resultar en tiempos de ejecución muy bajos, sin embargo estos ordenadores de propósito general en algunos casos pueden encontrarse con problemas debido al tamaño de la red neuronal.

La solución que se está aplicando hoy en día es el uso de GPUs, el cual son sistemas que tienen núcleos más pequeños y pueden realizar estas computaciones aún más rápido que las CPUs de propósito general, pero aun así no son la mejor opción para poder realizar este tipo de operaciones.

Por eso, Google ha inventado las TPUs, que son dispositivos cuyo hardware ha sido diseñado para ejecutar esta clase de redes neuronales de una manera más eficiente. Se pueden detallar las distintas partes de una TPU a continuación.

1. **Unidad multiplicadora de matrices.** Dentro de la TPU, existe una unidad que está optimizada para realizar grandes operaciones matriciales por medio de hardware. A diferencia de un ordenador de propósito general, en donde la unidad aritmética es de un propósito más general, esta unidad aritmética puede realizar operaciones de una manera más veloz, ya que no depende de instrucciones por parte del software, lo cual impacta el rendimiento de ejecución de la CPU.
2. **Unidad de procesamiento de vectores,** la cual mejora la capacidad de la TPU para realizar operaciones en vectores, lo cual simplifica las tareas como los calculos de funciones de activación. Igual que la multiplicación de matrices, esta unidad está diseñada en el hardware y es de uso específico para esta tarea, lo cual produce un mayor rendimiento que una CPU estándar ejecutando las mismas tareas.
3. **Memoria de gran ancho de banda.** A diferencia de un ordenador de propósito general con una memoria DRAM, que es más lenta, se encuentra ubicada en el exterior de la CPU y requiere circuitos y controladores extra, una TPU contiene una memoria SRAM, en el caso de la Coral Edge TPU es de 8 MB, la cual garantiza un mayor rendimiento y eficiencia, especialmente cuando se ejecutan modelos complejos y grandes volúmenes de datos.
4. **Interconexiones personalizadas.** A diferencia de una CPU de uso general, la TPU permite reconectar secciones internamente, permitiendo obtener un mayor rendimiento comparado con hacer uso de un bus compartido por distintos componentes. Además, en un entorno

cloud, donde se hace uso de TPUs más avanzados, podemos interconectar TPUs una con otra para obtener un mayor grado de paralelismo.

Finalmente, cabe destacar que en las TPUs, a diferencia que una CPU de propósito general o GPU, las funciones de activación también se realizan por medio del hardware, lo cual nos permite a velocidades superiores que se podría hacer si fuese en un entorno de software con un sistema operativo de por medio.

Este tipo de optimizaciones nos permiten obtener una mayor eficiencia energética, mayor escalabilidad, ya que se pueden añadir muchos chips a la red y que trabajen en conjunto, y un mayor rendimiento en general comparado con un sistema de propósito general o uno más específico como podría ser una GPU.

Detección de objetos

La detección de objetos es una técnica que hace uso de redes neuronales para localizar y clasificar objetos en una imagen, la cual hoy en día tiene muchos usos, como puede ser en la medicina o en la automoción.

Para detección de objetos existen multitud de modelos preentrenados para los distintos marcos de trabajo que existen, en este caso vamos a hacer uso de una gama de modelos llamada YOLO, que significa *You Only Look Once*, la cual es una familia de modelos desarrollado por Ultralytics código abierto diseñado para identificar objetos en imágenes. En este caso, YOLO v5 es una versión de un modelo en la familia y tiene 4 versiones distintas: *small*, *medium*, *large* y *extra large*, cada uno con mayor precisión que el anterior.

Primera toma de contacto

Para empezar a experimentar con TensorFlow en la *Raspberry Pi* con la *Raspberry Pi Camera*, vamos a empezar haciendo una copia del repositorio oficial de la *Raspberry Pi Camera*, ya que este trae algunos ejemplos que podemos seguir que hacen uso de TensorFlow. El que nos interesa se encuentra bajo la carpeta `examples/tensorflow/yolo_v5_real_time_with_labels.py`.

Antes de ejecutarlo tenemos que instalar las dependencias que necesita esta aplicación. Para instalarlo vamos a hacer uso de un entorno virtual de Python configurado de una forma específica, que le permite hacer uso de las librerías locales

```
python3 -m venv tflite --system-site-packages
source tflite/bin/activate
```

Sin añadir la flag `--system-site-packages` el sistema no va a poder hacer uso de la librería `libcamera`, que es necesaria para hacer uso de la *Raspberry Pi Camera* desde *Python*.

Una vez hecho, también tenemos que tener en cuenta que, partiendo desde 2024, Google dejó de soportar el componente `tfite-runtime`, el cual es necesario para poder procesar modelos de *TensorFlow Lite* usando *Python*. A cambio, han establecido que se tiene que usar el paquete `ai-edge-runtime`. Necesitamos hacer algunos cambios al script que trae el repositorio para poder ejecutarlo.

Con el conocimiento de todo esto en mente, procedemos a instalar las dependencias necesarias para poder hacer la inferencia.

```
sudo apt-get install python3-picamera2 python3-opencv  
pip3 install opencv-python opencv-python-headless picamera2 ai-edge-  
litert
```

Una vez tenemos estas dependencias, vamos a modificar el script un poco. En la línea 29 se cambiaría

```
import tflite_runtime.interpreter as tflite
```

por

```
import ai_edge_litert.interpreter as tflite
```

Esto va a cambiar la antigua librería que no se mantiene por la librería nueva.

También se va a hacer un cambio a la manera que el programa normaliza la imagen. La normalización es un proceso que ajusta los píxeles de una imagen a una escala estándar, la cual es más conveniente para un modelo de *machine learning* como lo es *YOLO v5*.

Para ello vamos a cambiar la línea

```
input_data = (np.float32(input_data) - 127.5) / 127.5
```

por este código a continuación

```
input_data = np.expand_dims(img, axis=0).astype(np.float32)  
input_data = input_data / 255
```

Esto va a hacer que los modelos que ejecutemos tengan mayor precisión y sean capaces de identificar objetos con más facilidad, además que va a permitir ejecutar modelos creados con una finalidad específica, la cual se va a explorar más adelante..

Una vez modificado, se podrá ejecutar este programa correctamente, y se procederá a probar el modelo que trae el repositorio por defecto.

Ejecución en CPU

Tras hacer las modificaciones pertinentes, nos podemos dar cuenta que ejecutar este modelo en una *Raspberry Pi 3B* base obtiene una velocidad muy reducida de inferencia, siendo incapaz de hacer reconocimiento de objetos en una imagen como se quería lograr en tiempo real.

La velocidad lograda por el sistema es en torno a 5 segundos por fotograma, tardando 5 segundos en refrescar la caja que indica el objeto señalado por el sistema.

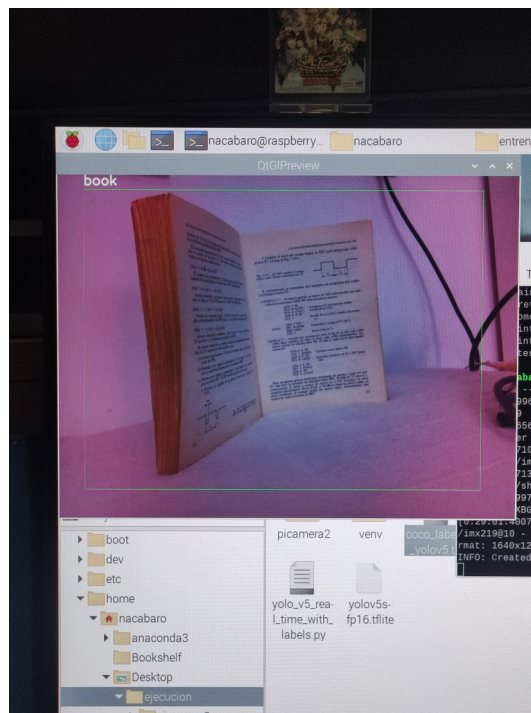


Figure 3: Detección de libro

También nos podemos percatar de que el uso de CPU se dispara al máximo, el dispositivo empieza a hacer un consumo elevado de electricidad, hasta el punto de empezar a indicar que existe deficiencia de corriente, además de una alta temperatura de CPU de 70 °C, lo cual puede reducir la vida útil del equipo si se mantiene la ejecución de esta forma.

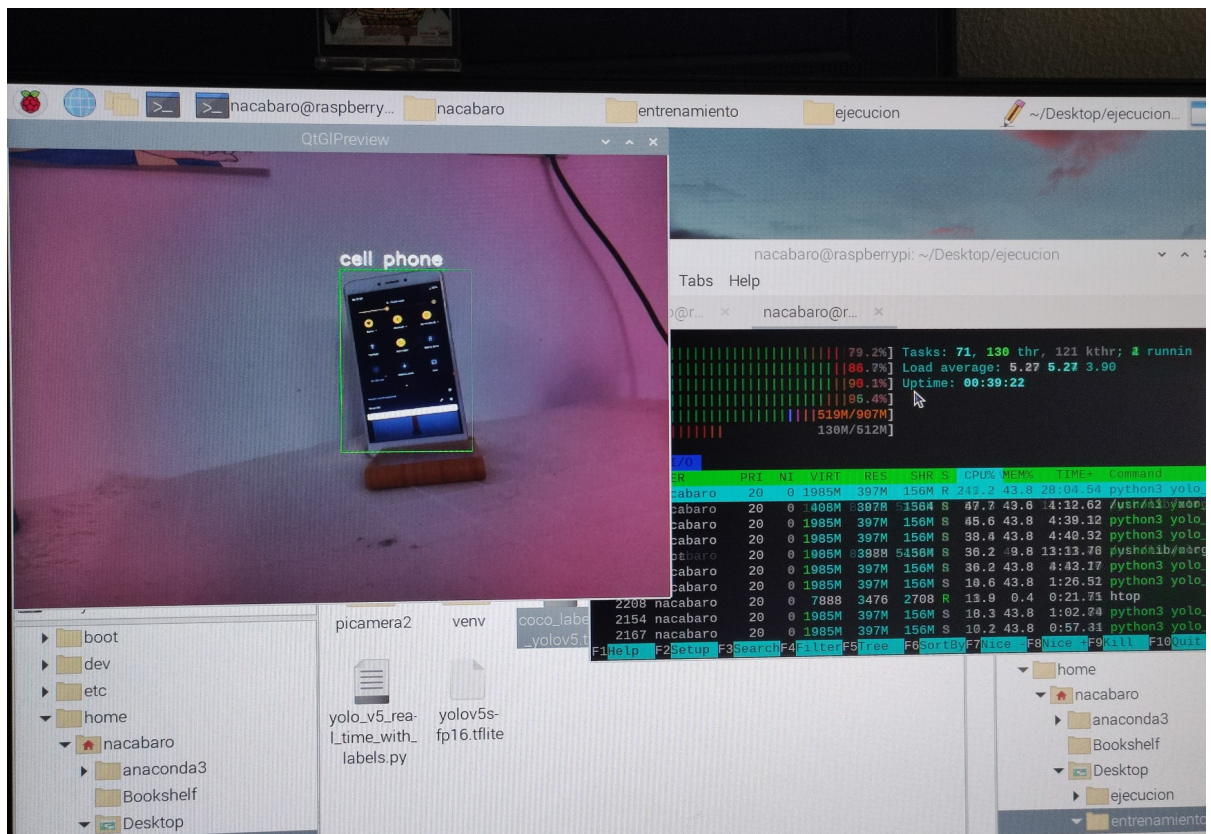


Figure 4: Detección de teléfono móvil y uso de CPU promedio

Además es importante destacar que esta falta de velocidad impacta a la precisión de los resultados, ya que no le da tiempo a actualizarse cuando se producen cambios.

Ejecución en TPU

Debido a las diferencias estructurales entre una CPU y la *Coral Edge TPU*, es necesario recompilar el modelo de tal forma que sea compatible con el dispositivo en cuestión.

Este proceso se le conoce como cuantización, el cual consiste en la reducción de la precisión del modelo con tal de hacerlo más pequeño y más rápido a la hora de tener que instalarlo en *hardware* con menor potencia.

Desde hace poco, este proceso se tiene que realizar en hardware más potente, y no en la *Raspberry Pi*, ya que Google dejó de soportar el compilador en arquitecturas que no sean las arquitecturas *x86_64*, además que es necesario de una CPU o GPU potentes.

Para ello, vamos a instalar el compilador en un sistema Linux de la siguiente forma. Este compilador acepta modelos TensorFlow Lite como el que trae el repositorio como ejemplo, en un formato FP16 o FP32, y los convierte a un formato que acepta la aceleradora.

```
curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | sudo apt-key add -  
echo "deb https://packages.cloud.google.com/apt coral-edgetpu-stable main" | sudo tee /etc/apt/sources.list.d/coral-edgetpu.list  
sudo apt-get update  
sudo apt-get install edgetpu-compiler
```

Una vez instalado, se procede a recompilar el modelo.

```
edgetpu-compiler yolov5s-fp16.tflite
```

Y esto generará un archivo llamado `yolov5-fp16-edgetpu.tflite`. En un caso ideal, un modelo se puede recompilar, no todos se pueden recompilar. Este modelo es uno de ellos que no se ha podido recompilar, entonces para realizar la ejecución de este se ha utilizado un modelo MobileNet V2

Segmentación de objetos

En común, la segmentación y la detección de objetos comparten el mismo objetivo, que es encontrar un objeto, la diferencia entre ambos tiene que ver con la precisión. Mientras que en la detección de objetos, normalmente el objeto solo se identifica por medio de un cuadrado, la segmentación consiste en enmascarar el objeto, aumentando la precisión de selección del area.

Algunas aplicaciones que se pueden ver diariamente de segmentación es en videollamadas. Pongamos un ejemplo en Google Meet, es posible enmascarar al sujeto del fondo, permitiendo ocultar el contenido de la habitación por medio de un fondo. Otras aplicaciones pueden consistir en cálculo de areas, como puede ser el area de un objeto comparado con el resto de la foto, si se conoce la distancia a la que se encuentra el sujeto de la cámara.

Para realizar la segmentación, existe la familia de modelos DeepLab y MobileNet, en este caso se hará uso de Deeplab.

Primeros pasos

Para proseguir con este modelo, se va a hacer uso del script obtenido en el repositorio de ejemplos incluido con la *Raspberry Pi Camera*. En este repositorio podemos ver un modelo muy básico de Deeplab V3 con precisión FP16.

Observando los elementos que se han utilizado para entrenarlo, podemos ver que objetos se pueden usar sin tener que entrenar el modelo de nuevo, en este caso he hecho uso de un bote de desodorante, ya que aparecía en la lista de objetos preentrenados

Antes de poder ejecutar el script, hay que actualizar las dependencias. Al igual que se ha hecho con el script para ejecutar YOLO v5, se procederá a hacer lo mismo en estas circunstancias, a excepción del ajuste de normalización.

También vamos a hacer uso del mismo entorno de *Python* utilizado para la demostración anterior, ya que hace uso de los mismos paquetes.

Finalmente, para realizar la ejecución del programa de Python se debe de hacer uso del siguiente comando:



Figure 5: Objeto a segmentar

Ejecución en CPU

Ejecutando el modelo incluido podemos darnos cuenta que este, al igual que los otros, se ejecuta de manera lenta, pero en comparación al modelo anterior, se ejecuta a mayor velocidad que el previamente mencionado YOLO v5.

Además podemos percatarnos de que la segmentación no es perfecta, esto podría darse debido a problemas de iluminación durante la ejecución, ya que al ser una cámara de visión nocturna, los experimentos se deben de realizar en un entorno muy controlado, ya que la luz solar puede provocar que la imagen obtenga un color rosado.

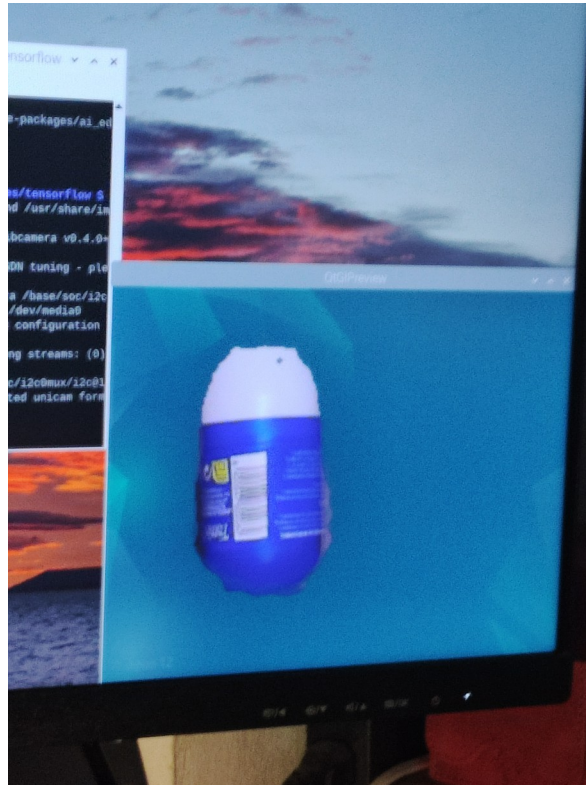


Figure 6: Extracción de un elemento del fondo

Igualmente podemos percatarnos de los mismos problemas de uso de recursos disparado (Figura 7) que conllevan ejecutar el modelo en la CPU: alto uso de CPU, alto consumo energético e incremento notable en la temperatura del sistema, lo cual, como se ha expuesto anteriormente, puede suponer en una degradación del hardware si se ejecuta de manera prolongada.

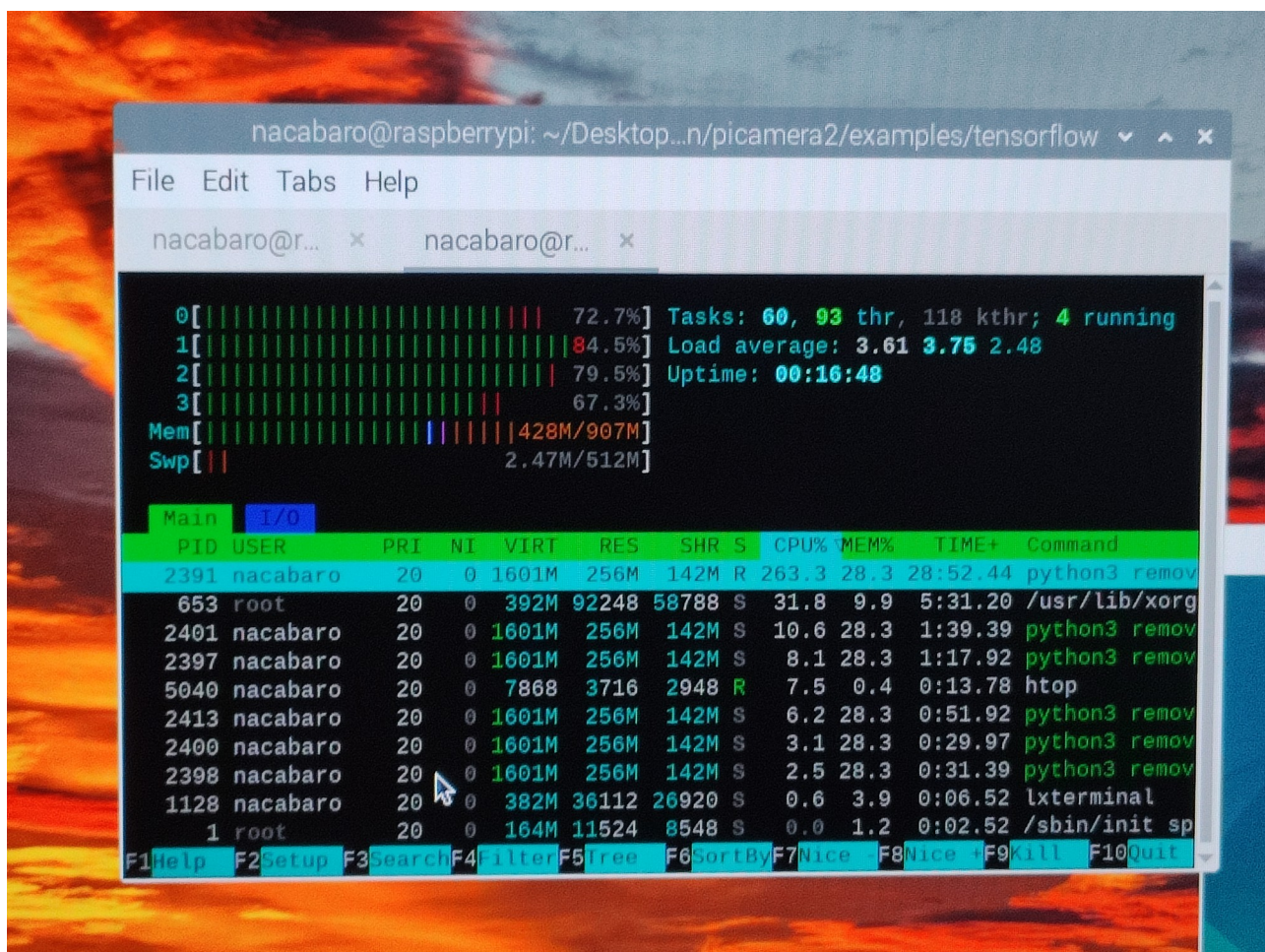


Figure 7: Uso de recursos durante ejecución de Deeplab V3

Benchmarking

Para poder evaluar el rendimiento de la TPU a diferencia de la CPU, he sometido el modelo a evaluar un video de 26 segundos entre distintos sistemas, entre los cuales se encuentra la anteriormente mencionada Raspberry Pi 3B, una CPU de portátil Ryzen 5 3500U y una Raspberry Pi 5. Los resultados de la evaluación son los siguientes:

Table 1: Tiempos de ejecución en distinto tipo de hardware

Sistema	Tiempo de ejecución
Ryzen 5 3500U (CPU)	86.77 segundos
Ryzen 5 3500U (TPU por USB 2.0)	75.45 segundos
Ryzen 5 3500U (TPU por USB 3.0)	53.34 segundos
Raspberry Pi 3B (CPU)	590.58 segundos
Raspberry Pi 3B (TPU) ¹	Violación de segmento
Raspberry Pi 5 (CPU)	47.37 segundos
Raspberry Pi 5 (TPU) ¹	Violación de segmento
Raspberry Pi 3B (TPU) ²	213.76 segundos
Raspberry Pi 5 (TPU) ²	39.13 segundos

¹Las librerías oficiales resultaban en una violación de segmento, la cual impedía ejecución en TPU. A día de hoy se desconoce la razón de por que produce este error.

²Esta se está ejecutando en una librería no oficial, que se puede obtener a partir de GitHub. Esta librería se encuentra pendiente de ser unida a la rama principal de la librería oficial, pero Google no ha tomado acción aún.

Este benchmark sigue haciendo uso de CPU, ya que se tienen que dibujar los cuadrados alrededor de los objetos detectados, esto se puede notar especialmente en la Raspberry Pi 3B, donde el tiempo de ejecución mejora por 376.82 segundos, que es un gran tiempo de mejora, y con más optimizaciones podría mejorar aún más.

También se puede observar una gran mejoría en el portátil Ryzen, donde se pasa a ejecutar un modelo de lo originalmente observado de 86.77 segundos a solo 53.34 segundos enviando la carga de trabajo a la TPU.

En general, podemos observar una mejora de los tiempos de ejecución sobre todo los dispositivos en los que se ha ejecutado la carga de trabajo.

Pipelining

Conociendo las ventajas e inconvenientes de cada modelo, podemos hacer uso de esta información proporcionada por ambos para poder calcular el area que puede ocupar un objeto con una mayor precisión siguiendo los siguientes pasos:

1. **Extracción del elemento de interés usando YOLO v5:** este modelo es más costoso de ejecutar, pero tiene una mayor precisión de detección de los objetos en una imagen, cuyo resultado luego se puede usar para recortar el objeto de interés de la imagen y almacenar de manera separada. Esto además nos permite guardar cierta telemetría del objeto a estudiar en cuestión, sin tener que almacenar el fotograma completo de la imagen, que luego puede ser extraída del sistema para evaluaciones ajenas al propósito entrenado, permitiendo reducción de uso de espacio.
2. **Extracción más precisa usando Deeplab v3 sobre los datos extraídos por YOLO v5:** De esta forma, si se conoce la distancia de la que se encuentra la cámara del objeto en cuestión, se puede llamar a este segundo modelo para que, a través de los datos extraídos, pueda hacer un cálculo del área ocupada por un objeto de interés. Además, tras haber extraído elementos que pueden provocar falsos positivos o anomalías causadas por detectar un objeto que no corresponde, se aumentaría la precisión de los datos obtenidos.

Debido a como funciona la memoria RAM que incluye la *Coral Edge TPU*, podemos ejecutar varios modelos de manera simultánea. También es posible aumentar la velocidad de la aceleradora, ya que para ahorrar energía se ejecuta a una velocidad baja. Pero cambiando el driver por otro también oficial, se puede incrementar la velocidad de ejecución a cambio de que la aceleradora incremente su consumo energético y un aumento considerable de la temperatura, lo cual en algunas situaciones puede considerarse no ideal.

Entrenamiento de modelo de detección

Para entrenar el modelo de segmentación se va a hacer uso de las siguientes herramientas

1. **Label Studio:** Herramienta que se va a utilizar para etiquetar las imágenes que luego se utilizarán durante el entrenamiento.
2. **Módulo Python venv:** Administrador de entornos de Python, que nos permitirá crear un entorno donde poder instalar todas las dependencias de Python, y es más recomendable que tener que instalar los paquetes en el sistema, ya que podemos tener conflictos sobre los paquetes ya instalados.
3. **YOLO v5:** Versión del modelo que se va a entrenar. Se puede obtener a través del repositorio oficial de YOLO. Existen modelos más nuevos y más potentes, como por ejemplo YOLO v12, pero para hacer pruebas he decidido usar YOLO v5, ya que sigue siendo un modelo muy competente hoy en día.

Introducción

El entrenamiento del modelo consiste en recopilar datos sobre el objeto que se quiere identificar, y hacer que el modelo sea capaz de reconocer los objetos que nos interesan. También se puede realizar *fine tuning*, que consiste en entrenar nuevos datos sobre el modelo ya entrenado, que en teoría es mucho más cómodo, y no necesitamos recopilar nuevos datasets.

Para esta demostración, voy a hacer un entrenamiento tal que el modelo solo sea capaz de reconocer un objeto de mi interés.

Etiquetación de imágenes

Para empezar tenemos que hacer varias fotos del sujeto, u objeto que ha de reconocer el modelo, siendo alrededor a 10 o 15 fotos por objeto a reconocer. Una vez hechas las fotos es necesario escalar todas las fotografías a 640x480, ya que el hardware sobre el que estamos trabajando no es de muy alta potencia, y va a ser más fácil de entrenar. Más adelante el programa de entrenamiento las escalará a 640x640, que es la resolución de entrenamiento del modelo que vamos a seguir, y también es la resolución que deberán de seguir las imágenes que se le pasen al modelo para que este pueda hacer detección de objetos.

Para entrenar este modelo, lo he entrenado para que solo pueda reconocer un objeto, para facilitar las demostraciones y reducir los falsos positivos durante las demostraciones. Luego he hecho alrededor de 25 fotografías de este objeto en distintas posiciones y distinta iluminación. A continuación, he redimensionado las imágenes de antemano antes de pasarlas por el programa de entrenamiento y por la clasificación.

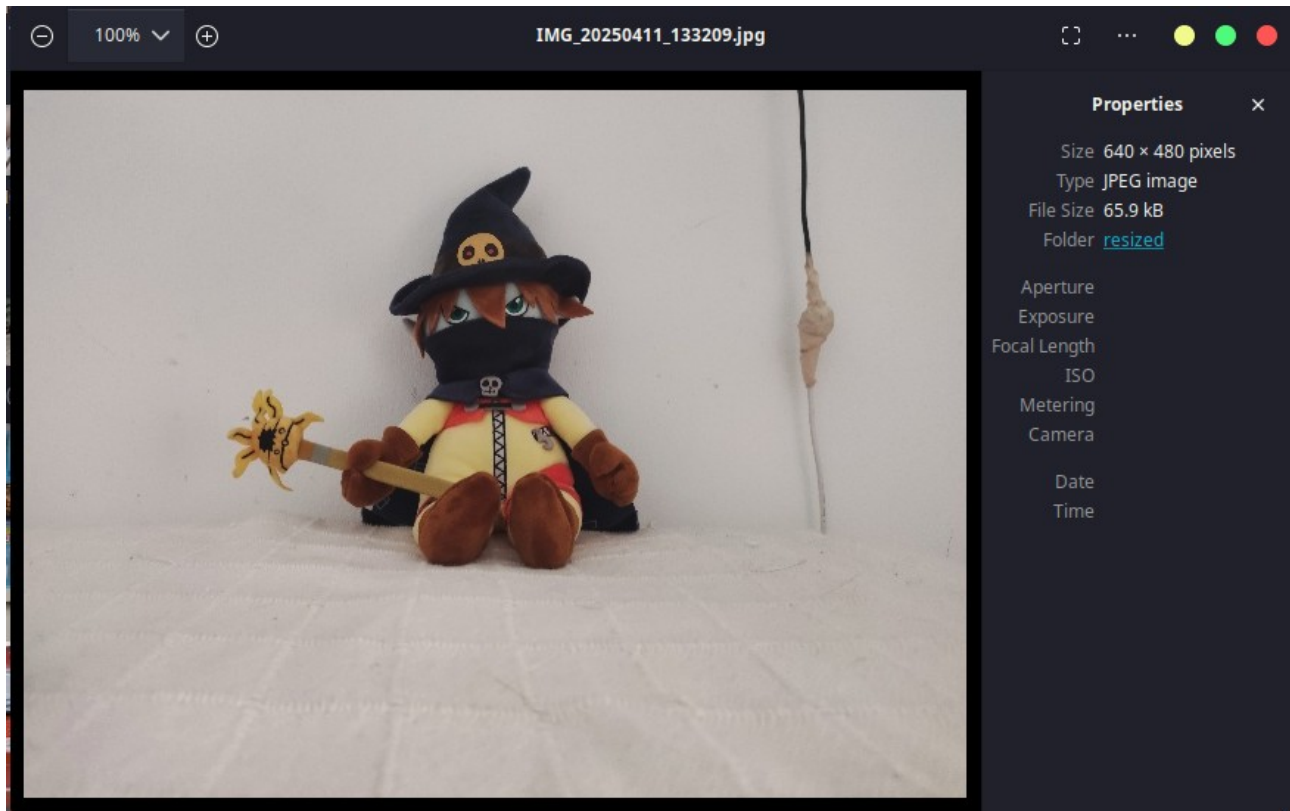


Figure 8: Ejemplo de imagen redimensionada y objeto a reconocer

A continuación, se deberá de abrir el directorio de imágenes escaladas con `Label Studio`, este nos permite de manera fácil categorizar los elementos dentro de las imágenes, y nos permite etiquetar los distintos objetos que se quieran identificar dentro de las imágenes.

Primero descargamos `Label Studio` en nuestro ordenador auxiliar, creamos una cuenta y luego creamos un nuevo proyecto. A continuación subimos todas las imágenes que hemos cambiado de resolución a 640x480 y simplemente importamos imágenes a `Label Studio` con todos los elementos que vayamos a identificar.

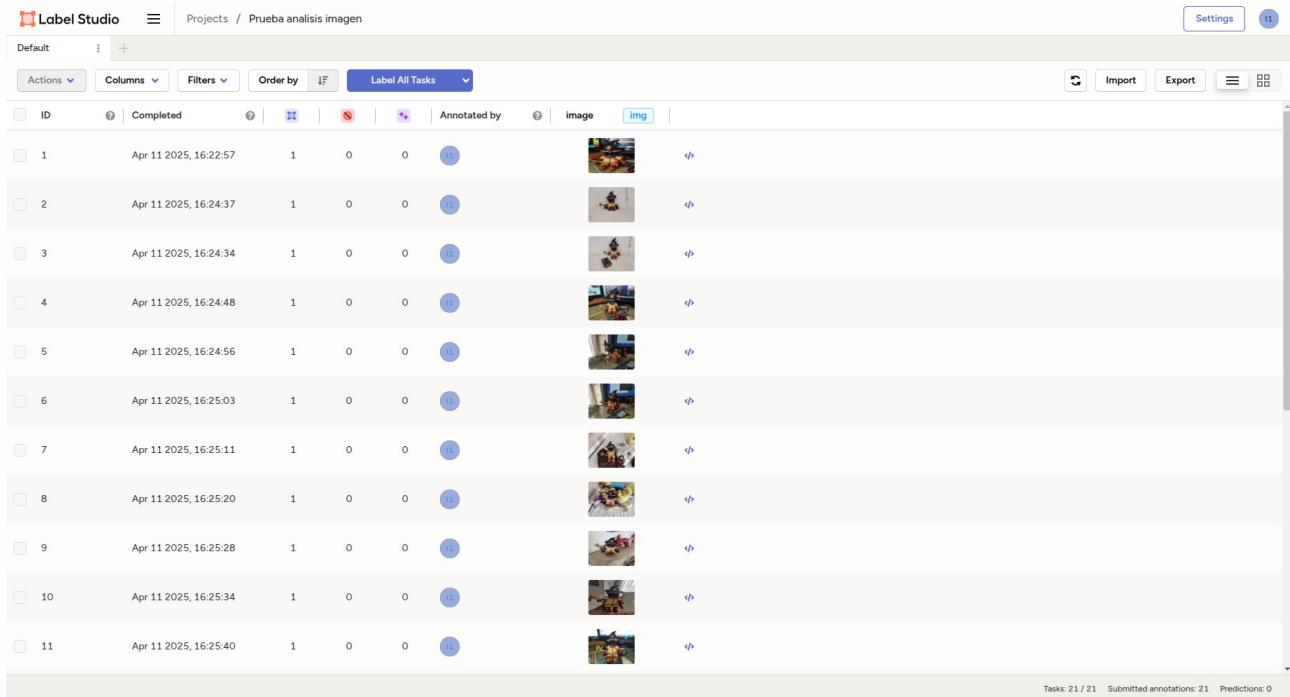


Figure 9: Interfaz de Label Studio con las imágenes cargadas

A continuación, se utilizará la función *Label All Tasks*, que a continuación nos obligará a crear una vista. Una vista es el diseño de la interfaz gráfica que se mostrará mientras se categorizan las imágenes. Existen diversas vistas, cada una diseñada para un modelo específico o tarea de etiquetación.

Cuando se abra la ventana de configurar vista, he introducido este código, que es un *template* básico para poder clasificar imágenes en el programa. Ya que este programa se puede usar para varios modelos distintos y tiene un alto grado de configuración. Este ejemplo lo he descargado de internet.

```
<View>
  <Image name="image" value="$image"/>
  <RectangleLabels name="label" toName="image"
model_score_threshold="0.25">
  <!-- Aqui se introducen los nombres de nuestras etiquetas -->
  <Label value="Car" background="blue"
predicted_values="jeep,cab,limousine,truck"/>
</RectangleLabels>
```

```
</View>
```

Tras haber configurado la vista, se puede añadir más etiquetas manualmente sin tener que modificar el código de la misma.

Después de configurar las etiquetas correspondientes a los objetos que se quieren etiquetar, se puede volver a la pantalla principal del proyecto, donde ya se puede empezar la tarea de etiquetación de objetos.

Para ello deberemos de colocar un rectángulo sobre el objeto que queremos identificar, en todas las imágenes de la forma que consta en la figura 10.

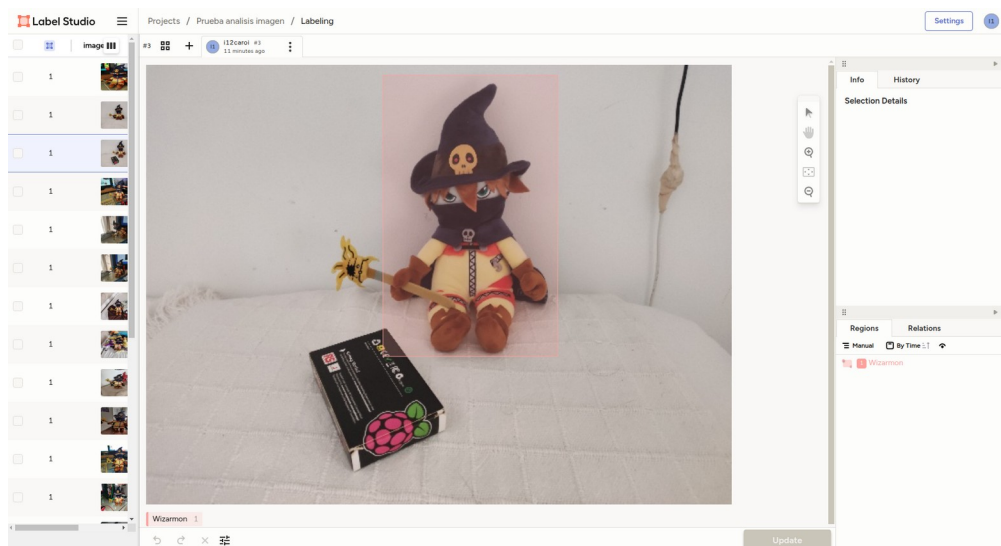


Figure 10: Ejemplo de añadir etiquetas a un objeto

Tras haber etiquetado todas las imágenes que van a formar el dataset, exportamos los datos obtenidos en un formato que el modelo que vamos a entrenar sea capaz de entender, como se puede observar en la figura 11.

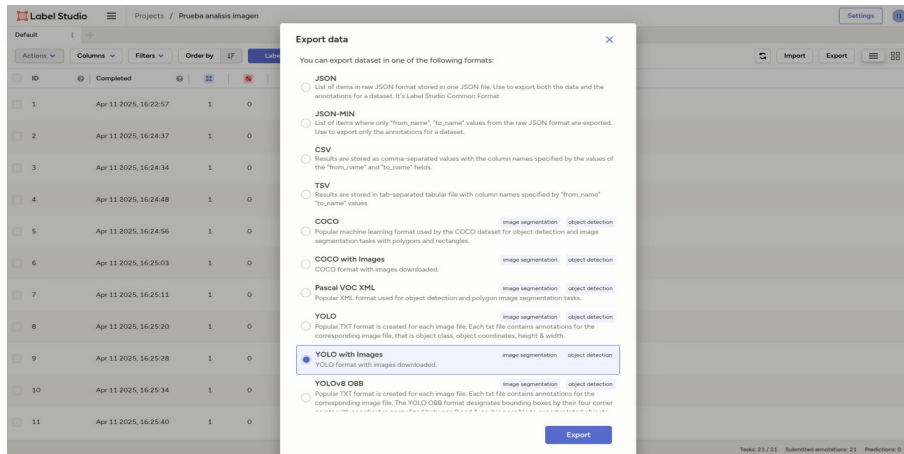


Figure 11: Exportación del dataset

Tras descargar el fichero Zip que contiene todas las imágenes más la información sobre las etiquetas, podemos proceder a realizar el entrenamiento del modelo.

Entrenamiento del modelo

Primero se procederá a descargar el repositorio de YOLO v5 en nuestro ordenador auxiliar, y vamos a configurar un nuevo entorno de Python 3 para instalar todas las dependencias.

```
git clone https://github.com/ultralytics/yolov5
python3 -m venv train
source venv/bin/activate
cd yolov5
pip3 install -r requirements.txt
```

Una vez estén todas las dependencias instaladas, se deberá de modificar el dataset, para ello en la carpeta de `data` dentro del repositorio tenemos varios ejemplos de como debe de estructurarse un dataset. Vamos a hacer un nuevo dataset, que deberá de ser codificado en formato YAML y tendrá que tener las siguientes opciones.

```
path: ../datasets/custom # dataset root dir (considerar que el
directorio donde se considera raiz es relativo al repositorio de yolo-
v5)
```

```
train: images # train images (relative to 'path') 128 images
val: images # val images (relative to 'path') 128 images
test: # test images (optional)

# Classes
names:
  0: Wizarmon
```

A continuación, se creará la carpeta donde se guardará las imágenes y metadatos para entrenar el modelo.

```
mkdir -p datasets/custom/images
```

Y en esta carpeta creada, se colocarán todas las imágenes correspondientes a nuestro modelo, incluyendo los metadatos de cada imagen. Se deberá de quedar la estructura de archivos de la siguiente manera.

```
$ ls .
yolov5 datasets
```

Es importante destacar que la carpeta donde se ubican los datasets se encuentra fuera del repositorio de git, así que se deberá de quedar la carpeta datasets fuera del repositorio, en el mismo directorio padre.

Dentro de la carpeta de `images` se deberá de tener la siguiente mezcla de archivos, que se puede observar en la figura 12.

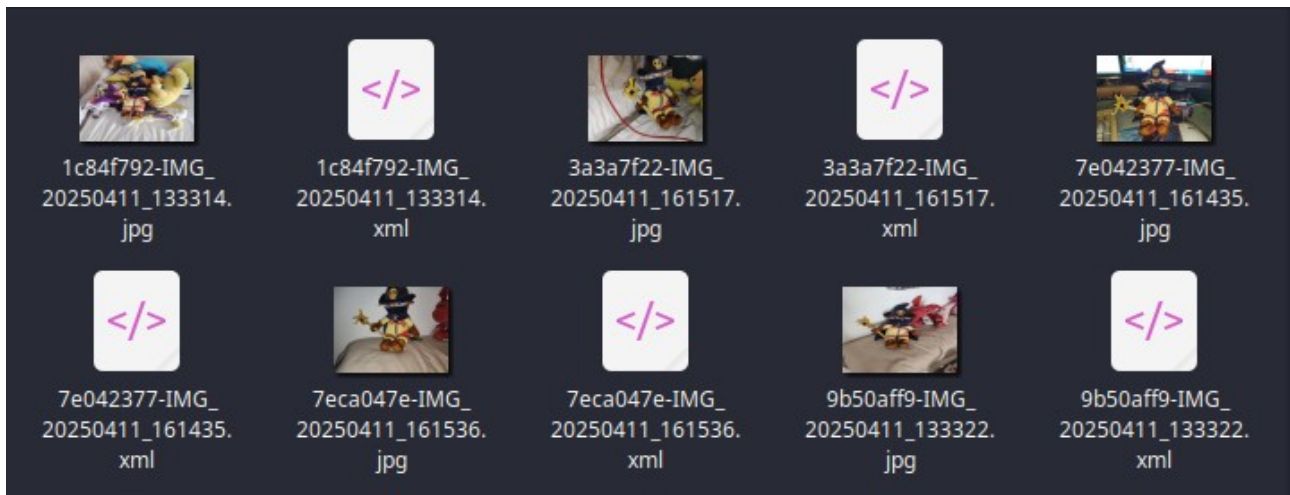


Figure 12: Contenido de la carpeta images

Finalmente, podemos proceder a entrenar el modelo ejecutando los siguientes comandos.

```
python3 train.py --img 640 --batch 16 --epochs 500 --data custom.yaml
--weights yolov5s.pt --nosave --cache
```

Para este caso vamos a usar YOLO v5, es un modelo menos complejo, pero debería de poder funcionar en sistemas de menores prestaciones. Una vez el modelo se haya entrenado, tendremos un modelo en formato PyTorch, el cual tiene que ser convertido a formato TensorFlow para luego ser compilado a un modelo de TensorFlow Lite, que será capaz de ser ejecutado en nuestra aceleradora.

Verificación del modelo

Una vez entrenado el modelo podemos verificar el funcionamiento del modelo si se ejecuta en nuestro ordenador auxiliar. Para ello tenemos que grabar un video donde aparezca este objeto que hemos entrenado, y podemos observar que el sistema es capaz de detectarlo.

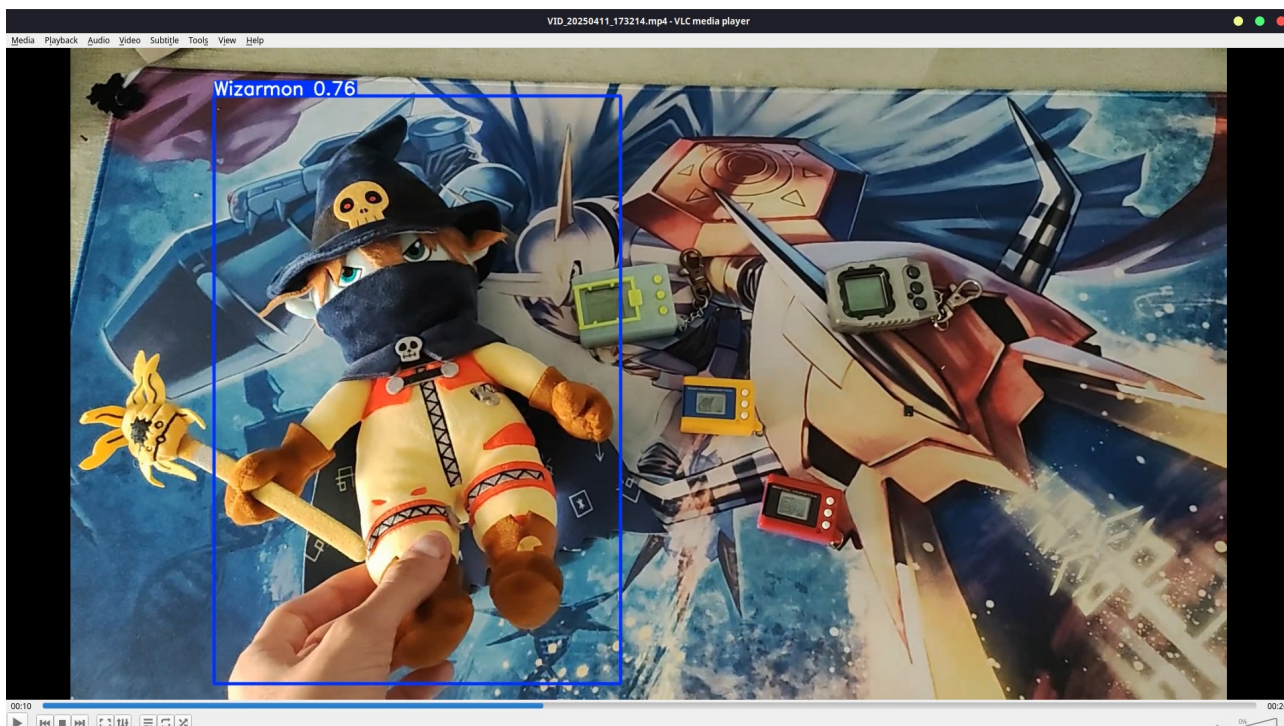


Figure 13: Detección del objeto usando PyTorch en ordenador auxiliar

Una vez verificado el modelo y que funciona como se espera, se puede proceder a recompilar el modelo para poder ejecutarse en el marco de trabajo de TensorFlow Lite.

Recompilar modelo

Antes de recompilar el modelo hace falta instalar TensorFlow en el ordenador auxiliar. Para eso, podemos modificar el archivo `requirements.txt` y descomentar la línea que contiene lo siguiente:

```
# tensorflow>=2.4.0,<=2.13.1 # TF exports (-cpu, -aarch64, -macos)
```

Y se vuelve a ejecutar el siguiente comando, que reinstalará las dependencias necesarias.

```
pip3 install -r requirements.txt
```

Para recompilar el modelo a TensorFlow Lite, YOLO v5 nos permite exportar el modelo usando el siguiente script, incluido en el repositorio del modelo.


```
python export.py --weights runs/train/exp2/weights/last.pt --include
tflite edgetpu
```

Este script generará dos archivos de salida, uno para TensorFlow Lite en formato FP16 y INT8. El formato INT8 está diseñado específicamente para las Edge TPU, mientras que el formato FP16 está diseñado para ser ejecutado por la CPU del dispositivo, ya que es un formato reducido que hace uso de menor cantidad de recursos.

Ejecución de este modelo personalizado

La ejecución se realizará de la misma forma que el modelo que se ha ejecutado anteriormente, solo hace falta cambiar el nombre del modelo en el parámetro del script. Es importante tener las dependencias instaladas en la *Raspberry Pi*, y podemos reutilizar el mismo entorno de Python que se configuró anteriormente.

En CPU

Ejecutando en CPU el modelo versión FP16, podemos ver que este funciona y es capaz de detectar el objeto que ha sido programado en tiempo real. Se sigue haciendo un uso elevado de CPU y el sistema alcanza una temperatura de 70.9 °C, lo cual no es una situación ideal, pero podemos verificar que la recompilación ha funcionado y que el modelo está funcionando.

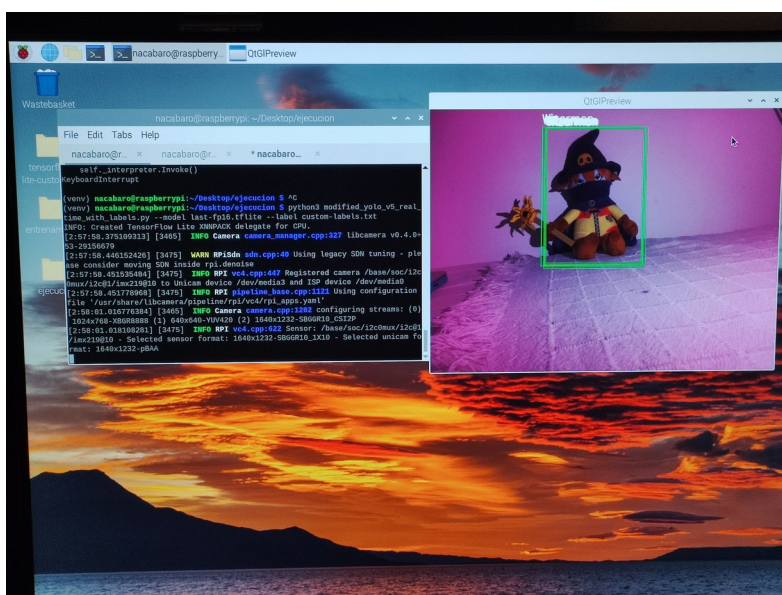


Figure 14: Detección del objeto programado en el hardware final

En TPU

Para ejecutar este modelo en TPU, modificaremos el script para añadir la siguiente línea.

```
delegate = load_delegate("libedgetpu.so.1")
```

Y finalmente se modificará la carga del interprete de TFLite para que haga referencia a este *delegate*

```
interpreter = tflite(  
    model_path=args.model,  
    num_threads=4,  
    experimental_delegates=[delegate]  
)  
interpreter.allocate_tensors()
```

Tras esto, se deberá de volver a cargar el script haciendo uso del modelo que ha sido precompilado para la TPU, ya que sino se seguirá ejecutando en CPU. Cabe destacar que existen modelos híbridos, en los que una parte se puede ejecutar en CPU y otra parte en TPU. En el caso de este modelo, solo se ejecuta en TPU.

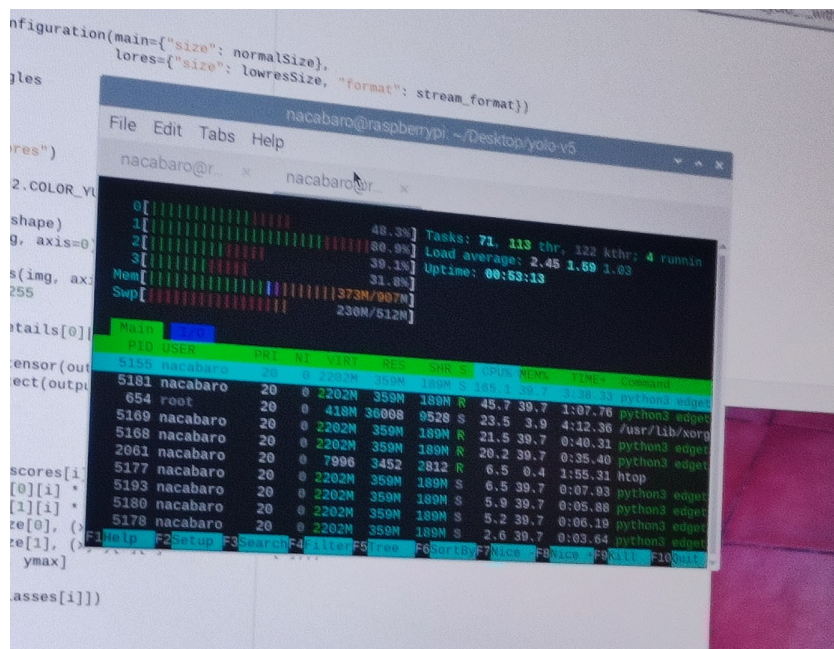


Figure 15: Uso de CPU durante ejecución del modelo en TPU

Sigue habiendo un uso de CPU un poco alto, pero se puede apreciar la reducción de uso al ejecutarlo en TPU. Este uso se puede explicar al tener que dibujar en pantalla los rectangulos, ya que es una tarea que hace un uso alto de CPU y tener que mostrar el video en directo en pantalla, que no es acelerado por el hardware.

Entrenamiento de un modelo de segmentación

Para el entrenamiento del modelo Deeplab V3 han hecho falta las siguientes herramientas:

1. **Anaconda:** Esta herramienta nos permite crear y gestionar entornos de Python virtuales, incluyendo poder establecer la versión de Python necesaria por el entorno. Debido a que es necesario usar TensorFlow 1.15.5 para entrenar este modelo, ha hecho falta instalar una versión de Python 3.6.
2. **TensorFlow:** Estas son las librerías necesarias para el entrenamiento del modelo y la conversión de este a TensorFlow Lite, además de crear los datasets en formato TFRecord.
3. **VIA v2:** VIA es una herramienta de etiquetación que nos permite de manera fácil etiquetar modelos, sobre todo los de segmentación, ya que hace falta etiquetar las imágenes de una manera específica.

Etiquetación de imágenes

Como se ha explicado anteriormente, la etiquetación de imágenes para este modelo se realiza de una manera completamente diferente al modelo mostrado previamente, ya que este modelo va a intentar recortar un elemento de la imagen, por lo que deberá de conocer el contorno del objeto a recortar.

Para esto, voy a hacer uso de la herramienta VIA. VIA es un etiquetador de imágenes muy simple, y además se ejecuta en el navegador. Podemos ver su interfaz en la figura 15. Para empezar se deberán de cargar las imágenes.

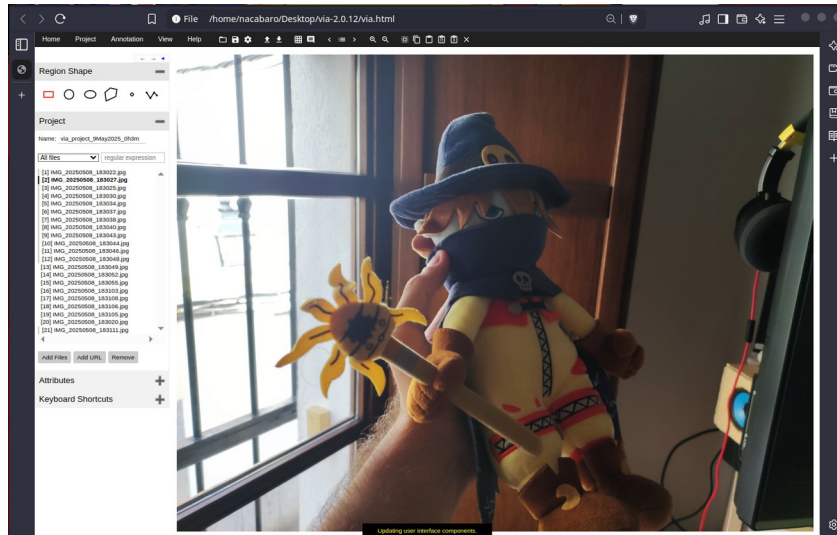


Figure 16: Interfaz de VIA

Una vez cargadas, se usará la herramienta *Polygon Region shape* y se seleccionará el contorno del objeto a segmentar, vease figura 16. Es importante ser lo más preciso posibles durante este proceso, ya que puede causar problemas al modelo.

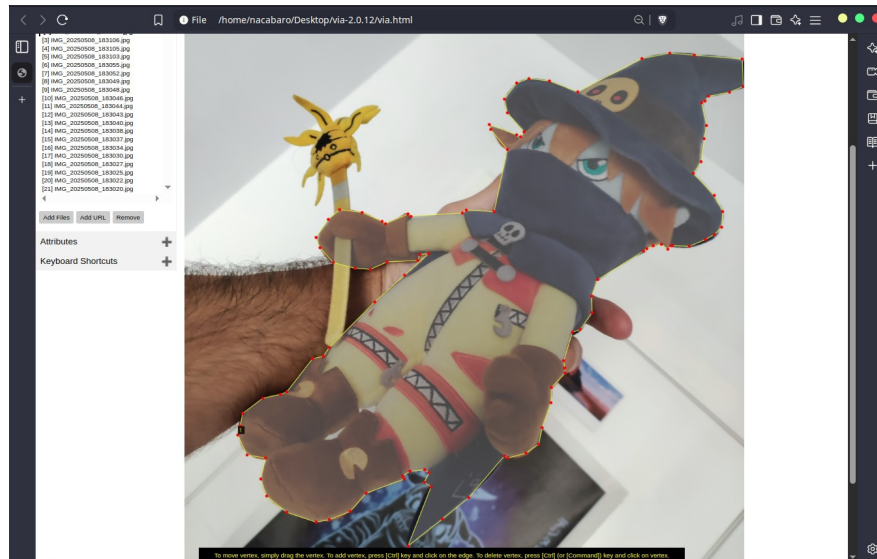


Figure 17: Objeto con contorno seleccionado

Tras seleccionar el contorno de todos los objetos que se quieran detectar, se deberá de guardar el proyecto. VIA exporta los proyectos en formato JSON. Este archivo JSON contiene una nube de puntos que es posible procesar usando Python y pillow para crear las imágenes con las máscaras. Para lograr esto he hecho uso de un script de Python que en base a las imágenes y la nube de puntos resultante genera las anotaciones en formato PNG.

Construyendo dataset en formato compatible

Para este proceso, el modelo incluye scripts que nos permiten descargar y convertir datasets públicos, como puede ser el Cityscapes, ADE20K y VOC2012. Ya que el dataset es personalizado, se deberá de modificar uno de los scripts. Primero es importante guardar las imágenes en el siguiente formato:

```
datasets/images/training # Imágenes originales, para el entrenamiento
datasets/images/validation # Imágenes originales, para la validación
datasets/annotations/training # Máscaras generadas por script de
Python, pero de las imágenes utilizadas durante el entrenamiento.
datasets/annotations/validation # Máscaras generadas por el script de
Python, pero para las imágenes utilizadas en la validación
```

Tras verificar que las imágenes están ubicadas en las ubicaciones correctas, ejecutaríamos el siguiente comando, dentro de la carpeta datasets del proyecto.

```
python ./build_ade20k_data.py \
  --train_image_folder="custom/images/training/" \
  --train_image_label_folder="custom/annotations/training/" \
  --val_image_folder="custom/images/validation/" \
  --val_image_label_folder="custom/annotations/validation/" \
  --output_dir="custom/output"
```

Esto generará los archivos necesarios para entrenar el modelo. Una vez generados, se deberá de especificar las propiedades del dataset en la lista de datasets. Para ello, se deberá de añadir lo siguiente en el archivo datasets/data_generator.py

```
_WIZARMON_INFORMATION = DatasetDescriptor(
    splits_to_sizes={
        'train': 21, # Numero de muestras en images/training
        'val': 21, # Numero de muestras en images/validation
        # se puede añadir un valor para hacer tests también
    },
    num_classes=2, # Numero de clases, en este caso son solo 2
    ignore_label=255,
```

```
)
```

Es importante declarar una clase más que la clase que se quiere identificar, ya que el fondo cuenta como una clase. Además también es importante establecer el número de imágenes en el repositorio. Finalmente, se añade el nuevo datasets al diccionario que contiene toda la información sobre los datasets, el cual se llama `_DATASETS_INFORMATION`.

```
_DATASETS_INFORMATION = {  
    'cityscapes': _CITYSCAPES_INFORMATION,  
    'pascal_voc_seg': _PASCAL_VOC_SEG_INFORMATION,  
    'ade20k': _ADE20K_INFORMATION,  
    'wizarmon': _WIZARMON_INFORMATION,  
}
```

Entrenamiento del modelo

Para empezar a entrenar el modelo, situados en la carpeta raíz del modelo, se procederá a ejecutar el siguiente comando:

```
python train.py \  
    --logtostderr \  
    --training_number_of_steps=1000 \  
    --train_split="train" \  
    --model_variant="mobilenet_v2" \  
    --output_stride=16 \  
    --train_crop_size="513,513" \  
    --train_batch_size=8 \  
    --base_learning_rate=3e-5 \  
    --dataset="wizarmon" \  
    --quantize_delay_step=0 \  
    --train_logdir=custom_log/ \  
    --dataset_dir=datasets/custom/output/
```

Es importante destacar las siguientes carpetas:

- `train_logdir`: Esta carpeta almacena los checkpoints generados durante el entrenamiento. Tras el entrenamiento estos checkpoints deberán de ser convertidos a un formato que pueda utilizar TensorFlow.
- `dataset_dir`: Esta carpeta es la salida generada por el script de creación de datasets. Importante apuntar a la carpeta output con los ficheros TFRecord, y no a las imagenes JPG y PNG.

Este proceso de entrenamiento es mucho más lento que el de YOLO, y solo se puede hacer en CPU, debido a la antigüedad de las librerías de TensorFlow, ya que necesita CUDA 10, y hoy en día solo se puede encontrar CUDA 12.

Además, es importante destacar que los parámetros establecidos durante el entrenamiento pueden resultar no ser óptimos. Este modelo tiene muchos más parámetros que el anteriormente mencionado YOLO v5, y se han establecido parámetros suficientes para entrenar una sola clase.

Una vez finalizado el entrenamiento, dentro de la carpeta `custom_log`, podremos observar los checkpoints realizados durante el entrenamiento, pero sobre todo el checkpoint 1000 es el de mayor interés. La ventaja del entrenamiento por checkpoints es que se pueden tener diversas versiones del modelo y se pueden hacer pruebas sobre las distintas versiones que se han generado del modelo durante el entrenamiento, en caso de encontrar un problema con overfitting.

Conversión de checkpoint a protobuf

Este paso generará los pesos finales del modelo y los almacena en un formato el cual TensorFlow puede manejar. Para ello, se nos incluye un script en el repositorio que es el de exportación de modelo.

```
python3 export_model.py \  
    --checkpoint_path=custom_log/model.ckpt-912 \ ## Este sería el  
checkpoint generado durante el entrenamiento  
    --quantize_delay_step=0 \  
    --export_path=custom_output/frozen_inference_graph.pb ## Lugar  
donde guardar los pesos ya exportados
```

Una vez obtenidos los pesos, podemos ejecutar pruebas del modelo directamente sobre el modelo mientras está en el estado de TensorFlow.

Conversión de protobuf a TensorFlow Lite

Para hacer la conversión a TensorFlow Lite, se incluye un script dentro del repositorio el cual recoge el fichero protobuf y convierte a TensorFlow Lite. Además nos pide utilizar una imagen, que se utilizará para realizar una prueba al modelo.

```
python3 convert_to_tflite.py \  
  --quantized_graph_def_path=${OUTPUT_DIR}/frozen_inference_graph.pb \  
  --input_tensor_name=MobilenetV2/MobilenetV2/input:0 \  
  --output_tflite_path=${OUTPUT_DIR}/frozen_inference_graph.tflite \  
  --test_image_path=imagen_prueba.jpg
```

Primero, se ha de importar el archivo generado por el paso anterior, ya que estos son los pesos del modelo ya entrenado. A continuación, se debe de especificar el nombre del tensor de entrada. Como este modelo hace uso de MobileNet V2 como backend, es importante especificarlo, ya que existen diversos backends para este modelo, la diferencia que existe entre uno y otro es que MobileNet V2 es el que mejor está optimizado para uso en EdgeTPUs. Luego se puede especificar una imagen para probar el modelo, la cual es una imagen de prueba que recomendablemente no se haya utilizado durante entrenamiento o validación.

Problemas encontrados

Debido a limitaciones técnicas, no se ha podido generar un modelo util a través de este proceso. Deeplab V3 no se ha actualizado en mucho tiempo, por lo que es necesario utilizar librerías obsoletas y versiones antiguas de Python, lo cual no es recomendable de cara a proyectos en el futuro, y ha generado muchos problemas durante el entrenamiento.

Esta limitación que traen las librerías antiguas han impedido también el uso de GPU, ya que hace falta una versión de CUDA más antigua por las limitaciones de TensorFlow 1.15.5, que es la última versión que soporta este modelo, ya que a partir de TensorFlow 2.0 cambiaron muchas funciones y actualmente, están deprecadas.

No se descarta la opción de actualizar el modelo, este proceso involucraría modificar el código para que haga uso de las librerías modernas que trae TensorFlow, y esta falta de actualización de los códigos de ejemplo no significa que Google este cerrando TensorFlow, ya que a día de hoy se siguen introduciendo versiones modernas de TensorFlow, y recientemente han

introducido una nueva librería para dispositivos como la Edge TPU, llamada LiteRT, que es el sucesor de TensorFlow Lite, y que se siguen versiones nuevas a día de hoy.

Otra opción disponible es usar un modelo distinto. Tras investigar, existen alternativas como pueden ser YOLO V11, el cual es un modelo que permite hacer segmentación y detección de manera simultánea, y se puede recompilar a TensorFlow Lite de manera fácil, pero este requiere una reescritura de los scripts de ejecución para que sean capaces de trabajar con esta información.

Otro modelo que se puede aplicar en el caso de segmentación es U²Net, el cual es un modelo que fue inventado para TensorFlow y únicamente puede hacer segmentación, aunque este es más difícil de manipular y hay falta de documentación.

Ejecución de modelos en NPU Hailo 8

Además de las ya exploradas Coral Edge TPU, existen más aceleradoras en el mercado. Este es el caso de la compañía Hailo AI, la cual introdujo una NPU en el mercado hace poco, y que ha intentado acercarse al público general gracias a la accesibilidad que proporciona el hardware *Raspberry Pi*.

Tras la introducción de la Raspberry Pi 5 con un bus PCI Express accesible por el usuario, se puede acceder a un bus de mayor velocidad que el USB que existía en la Raspberry Pi, en este caso un bus PCI Express 3.0 x4. Esto permite hacer más cosas con el hardware, ya que ahora se pueden acoplar periféricos como SSDs, antenas inalámbricas, y en este caso, aceleradoras como la mencionada Hailo, incluso compatibilidad con algunas GPUs de AMD.

En el caso de la Hailo 8, es una aceleradora NPU que tiene como entorno de desarrollo la librería Hailo RT, que se podría equiparar con el LiteRT, que es la que usa la aceleradora Coral Edge TPU. Además, ambos dispositivos no hacen uso del mismo tipo de modelos. Mientras que la Edge TPU hace uso de modelos en formato tflite, la aceleradora Hailo hace uso de su propio tipo de modelos.

Además del entorno de desarrollo, a principios de mayo de 2024, Hailo AI introdujo su nuevo Hailo Dataflow Compiler. Este nos permite compilar modelos de TensorFlow, PyTorch, ONNX, al formato que entiende la librería HailoRT.

Para poder evaluar el rendimiento de este dispositivo y poder realizar las pruebas pertinentes, es necesario cambiar el hardware que tenemos actualmente. Las cargas actuales se están ejecutando en una Raspberry Pi 3B, pero para hacer uso de la Hailo 8 es necesario una Raspberry Pi 5, con el bus PCI express, ya que no tiene otra forma de conexión.

Dataflow Compiler

Como se ha mencionado anteriormente, el software Dataflow Compiler nos permite convertir modelos de un formato, como PyTorch, TensorFlow, TensorFlow Lite, ONNX, entre otros, al formato nativo que nos ofrece la aceleradora Hailo.

Actualmente, el acceso a este compilador está restringido a empresas y a instituciones, y se puede solicitar el acceso a este mismo desde la página web de la compañía por medio de la Hailo

Developer Zone. He decidido no explorar este compilador debido a falta de tiempo y la dificultad de acceso al mismo.

Evaluación de hardware Hailo

Primero es necesario instalar la placa de la aceleradora sobre la Raspberry Pi 5. Para ello el kit nos trae unos espaciadores y tornillos para poder montar la placa a la Raspberry. Una vez montada, se debe de conectar la placa por medio de PCI Express, a través del cable que se incluye con la aceleradora.

También es importante destacar que es recomendable utilizar una fuente de corriente oficial de Raspberry Pi para hacer uso de la aceleradora, ya que el sistema requiere para funcionar una fuente de alimentación de 5 voltios y 5 amperios mínimo. No obstante, se puede ejecutar en una fuente de alimentación de 5 voltios a 2 amperios, pero a veces la aceleradora no es detectada por el driver de Hailo, y si se deben de conectar más periféricos, esto podría causar problemas.

Una vez instalado el sistema operativo, se deberá de actualizar el bootloader de la Raspberry Pi 5, ya que es un periférico muy reciente. Para ello se puede hacer por medio de raspi-config y el administrador de paquetes del sistema.

```
sudo apt update && sudo apt full-upgrade  
sudo rpi-eeprom-update
```

Si se ve una versión mayor a la del 6 de Diciembre de 2023, se puede proceder con la instalación de la placa, sino se deberá de actualizar el bootrom.

A continuación, para poder continuar con la configuración del acelerador, se va a instalar todas las dependencias relacionadas con la aceleradora Hailo. Estas se incluyen en los repositorios oficiales de Raspberry Pi y se pueden instalar de la siguiente manera.

```
sudo apt install hailo-all
```

Finalmente, se deberá de reiniciar el sistema y podremos probar si funciona la aceleradora ejecutando el siguiente comando

```
hailortcli fw-control identify
```

De esta forma podremos ver que aceleradora se encuentra conectada al sistema. Si la ejecución del comando falla, significa que la aceleradora puede que no esté conectada de forma correcta o que haya una deficiencia por parte de la fuente de corriente. En el caso de no tener la fuente de alimentación recomendada, se puede conseguir hacer que funcione el dispositivo reiniciándolo varias veces hasta que el comando devuelva una salida coherente, lo cual no es recomendable, ya que puede hacer que el sistema se apague debido a deficiencias.

A continuación vamos a obtener los ejemplos de Raspberry Pi preentrenados y se van a ejecutar pruebas sobre estos modelos. Para ello se procederá a clonar el siguiente repositorio:

```
https://github.com/hailo-ai/hailo-rpi5-examples
```

Una vez clonado, se instalará el entorno virtual de Python correspondiente por medio del siguiente comando.

```
chmod +x install.sh  
./install.sh  
source setup_env.sh
```

Finalmente, podemos ejecutar el primer ejemplo de detección, para ello hacemos uso de la siguiente herramienta que nos incluye el repositorio.

```
python basic_pipelines/detection.py
```

En el caso de querer ejecutar el procesamiento usando la cámara de la Raspberry Pi, podemos añadir el parámetro `--input rpi`. Una vez ejecutado, se deberá de ver un feed de video el cual está siendo ejecutado en la Hailo.

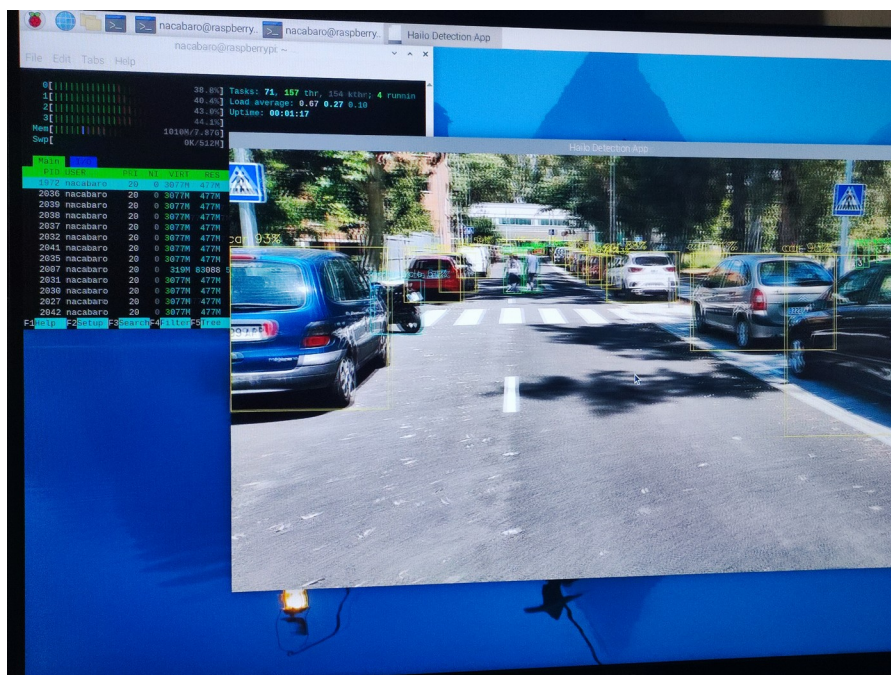


Figure 18: Ejecución de modelo de detección YOLO v6 en Hailo 8L

Se puede observar el código que está realizando el procesamiento en el fichero

```
venv_hailo_rpi5_examples/lib/python3.11/site-packages/  
hailo_apps_infra/detection_pipeline_simple.py
```

Podemos observar que la velocidad de procesamiento es mucho más rápida que TensorFlow Lite en CPU y en TPU por los scripts mostrados anteriormente, y que el uso de CPU es mucho menor. Como se ha descrito anteriormente, al tener que estar renderizando en pantalla, el uso de CPU incrementa.

Además de este modelo, se incluyen ejemplos de modelos de segmentación de instancias, detección de poses, y detección de profundidad. Podemos obtener más modelos preentrenados u otras pipelines desde el model zoo de Hailo. Aquí también se pueden encontrar formas de entrenar nuevos modelos específicos para la aceleradora hailo, pero solo si se tiene acceso al dataflow compiler.

Conclusión

El hardware que provee Hailo es mucho más potente que el que provee Google, como se puede observar en los ejemplos mostrados. La inferencia en este hardware se está ejecutando a tiempo real con bajo uso de recursos, el cual es el objetivo que se está buscando.

También cabe destacar el coste de esta solución, el cual es inferior a la competencia, en este caso siendo Nvidia con la Nvidia Jetson, que aproximadamente es de 349 euros actualmente.

Además, comparado con Google, el soporte a este hardware es mucho mejor, se puede utilizar en distribuciones de Linux modernas con versiones de Python modernas, mientras que el hardware ofrecido por Google tiene más problemas debido a repositorios que no se han actualizado en mucho tiempo y ejemplos que han tenido que ser modificados para que puedan funcionar.

Finalmente, cabe destacar que la empresa Hailo es mucho más cerrada. Es decir, mucha información, como puede ser herramientas, soporte, tutoriales y guías de entrenamiento no están disponibles al público, sino que tienes que ponerte en contacto con la empresa y revelar el propósito con el que va a ser utilizado el hardware antes de que puedas obtener acceso a las herramientas necesarias. Esto es completamente opuesto a Google, que provee herramientas, drivers, tutoriales e investigaciones de manera abierta y pueden ser utilizadas por cualquiera.

Aplicaciones

Este sistema tal como ha sido presentado, da un alto grado de libertad, ya que debido a sus especificaciones se puede montar en cualquier parte y se puede usar para recopilar datos y a su vez procesar los datos de manera local. De esta forma se puede instalar en lugares remotos y hacer el procesamiento en el lugar sin necesidad de tener una conexión fija a internet, ni tener un sistema de alta potencia.

Algunas aplicaciones que puede tener este sistema son las siguientes:

1. **Trabajo de campo.** Un sistema de estas prestaciones se puede usar para poder identificar anomalías en un lugar cuyo acceso sea muy difícil, como por ejemplo en un bosque o en algún lugar donde no se puede acceder regularmente para extracción de datos.
2. **Medicina.** Un sistema de estas prestaciones puede ser incorporado a maquinaria que tenga que ser transportada de manera constante, y poder operar identificando síntomas en base a rasgos físicos en el sujeto. Uno de estos casos podría ser un endoscopio portátil que sea capaz de identificar problemas en los pacientes.
3. **Transporte.** En el sector de la automoción, cada vez se está viendo el incremento en los coches que se pueden conducir de manera autónoma. Un sistema como estos puede ser muy útil para determinar límites de velocidad o señalización.

Posibles mejoras

Hoy en día cada vez se puede notar más la necesidad de poder realizar cómputo de *machine learning* en dispositivos con poco poder de procesamiento. Esto inició la creación de la *Coral Edge TPU*, pero desde entonces han surgido nuevas compañías, como la anteriormente mencionada Hailo AI.

Finalmente, cabe destacar que desde hace poco, fabricantes como Intel, AMD, Qualcomm o Apple, están instalando chips diseñados para acelerar cargas de IA y ML en los dispositivos como ordenadores, móviles o tablets, lo cual permiten cosas como por ejemplo, mejorar la seguridad a través de reconocimiento facial, predecir patrones del usuario, o incluso ayudar a las personas con discapacidad permitiendo usar el teléfono como una herramienta de accesibilidad.

Conclusión

El objetivo final de este trabajo es demostrar que hoy en día, existen soluciones para ejecutar modelos de inteligencia artificial de manera eficiente en dispositivos finales, sin tener que depender de sistemas complejos como pueden ser servidores con muchas GPUs o uso de altos recursos energéticos para realizar ciertas cargas de inteligencia artificial.

Ambos sistemas explorados, tanto el Hailo como la Coral Edge TPU poseen propiedades que los benefician para una carga de trabajo específica u otra, pero ambos realizan las cargas de manera correcta, y obtienen resultados que refuerzan la idea principal de esta investigación.

También cabe destacar las inconveniencias que traen ambos sistemas también, y que demuestra que aún queda mucho hasta que se pueda tener un sistema de esta escala como viable, ya que como se ha explorado, no existe un marco de trabajo estandarizado, o que no todos los modelos mencionados se pueden ejecutar en estos dispositivos sin tener que ser modificados de alguna forma u otra.

Finalmente, se puede observar como se está empezando a notar el interés de grandes fabricantes en esta industria, ya que hoy en día podemos encontrar dispositivos como TPUs en teléfonos móviles de Google, las cuales están diseñadas para ejecutar tareas relacionadas con el procesamiento de imágenes o detección de patrones del usuario. Además, podemos empezar a ver estos dispositivos incluidos en CPUs de Intel y AMD, e interés por parte de compañías como Microsoft, que están introduciendo estos dispositivos en Windows para poder también identificar patrones del usuario.

Para concluir, la estandarización de la forma de interactuar con estos dispositivos ofrecerá una gran capacidad a los usuarios de poder utilizar un único framework para poder interactuar con distintos dispositivos. Google intentó hacer esto usando TensorFlow Lite, el cual soporta carga de drivers específicos para ejecutar modelos TFLite en distintos tipos de dispositivos.

Glosario

- LiteRT: Anteriormente conocido como TensorFlow Lite, es un framework que permite la ejecución de modelos de machine learning en formato tflite. Este es un runtime, es decir, esta librería solo permite ejecutar modelos.
- HailoRT: Framework que permite ejecutar modelos de machine learning en el hardware de Hailo. Igual que TensorFlow Lite, este solo permite la ejecución de modelos.
- Model Zoo: Repositorio de modelos ya entrenados y quantizados por otros usuarios, habitualmente incluyendo un script que permite la ejecución del mismo.
- Pipelining: Sistema que nos permite añadir módulos para obtener una o varias salidas a partir de una entrada.

Bibliografía

<https://coral.ai/docs/accelerator/get-started>

<https://coral.ai/models/>

<https://www.raspberrypi.com/documentation/accessories/ai-kit.html>

<https://github.com/raspberrypi/picamera2/tree/main/examples/tensorflow>

<https://www.techtarget.com/whatis/definition/tensor-processing-unit-TPU>

<https://qengineering.eu/google-corals-tpu-explained.html>

https://github.com/hailo-ai/hailo_model_zoo

<https://github.com/hailo-ai/hailo-rpi5-examples/>

<https://github.com/tensorflow/models>

<https://github.com/ultralytics/yolov5>

<https://github.com/tensorflow/models/tree/master/research/deeplab>

<https://www.youtube.com/watch?v=oFNKfMCGiqE>

<https://www.youtube.com/watch?v=HgIMJbN0DS0>

<https://github.com/feranick/libedgetpu> (Repositorio con librería funcional)